STACK GUARD

Purpose and stack model

StackGuard provides boundary checks for a full-descending stack. PUSH decreases the stack pointer (SP) by one word, while POP increases it. The guard is constructed with stack_base and stack_limit and reports an INVALID_MEMORY_ACCESS exception when a checked pointer or operation violates the configured boundary.

STACK POINTER BOUNDS: stack_limit ≤ SP ≤ stack_base

VALID STACK ADDRESSES: [stack_limit, stack_base)

Interface and behavior

Method	Behavior
checkBounds(sp)	Validates the stack configuration and checks that SP is within the inclusive pointer bounds. Returns INVALID_MEMORY_ACCESS if stack_limit > stack_base, SP > stack_base or SP < stack_limit.
checkPush(current_sp)	Checks the prospective pointer before PUSH: rejects current_sp == 0, then rejects current_sp − 1 < stack_limit. Otherwise returns no exception.
checkPop(current_sp)	Rejects POP when current_sp ≥ stack_base; otherwise returns no exception.
isValidAddress(address)	Returns true only for addresses in the half-open stack-memory interval: stack_limit ≤ address < stack_base.
stackBase() / stackLimit()	Return the configured upper and lower stack boundaries, respectively.

Example boundary checks

`StackGuard guard(16, 0);` SP values 0 through 16 pass checkBounds; address 0 and 15 are valid, while address 16 is not. PUSH at SP=1 is permitted (new SP=0), but PUSH at SP=0 is rejected. POP at SP=15 is permitted; POP at SP=16 is rejected.

Validation and project files

Implementation: memory_safety/include/memory_safety/stack_guard.h and memory_safety/src/stack_guard.cpp. Tests: tests/memory_safety_test.cpp cover valid bounds, address-range endpoints, PUSH/POP boundary protection and invalid stack configuration. The test program reports “memory_safety_test: all checks passed” when no checks fail.

Example overflow scenario

examples/stack_overflow.asm recursively CALLs itself without a base case, saving R14 with PUSH at each level. As the stack grows downward beyond available memory, the expected simulator result is EXCEPTION: INVALID_MEMORY_ACCESS.